



FABRIC ATLAS

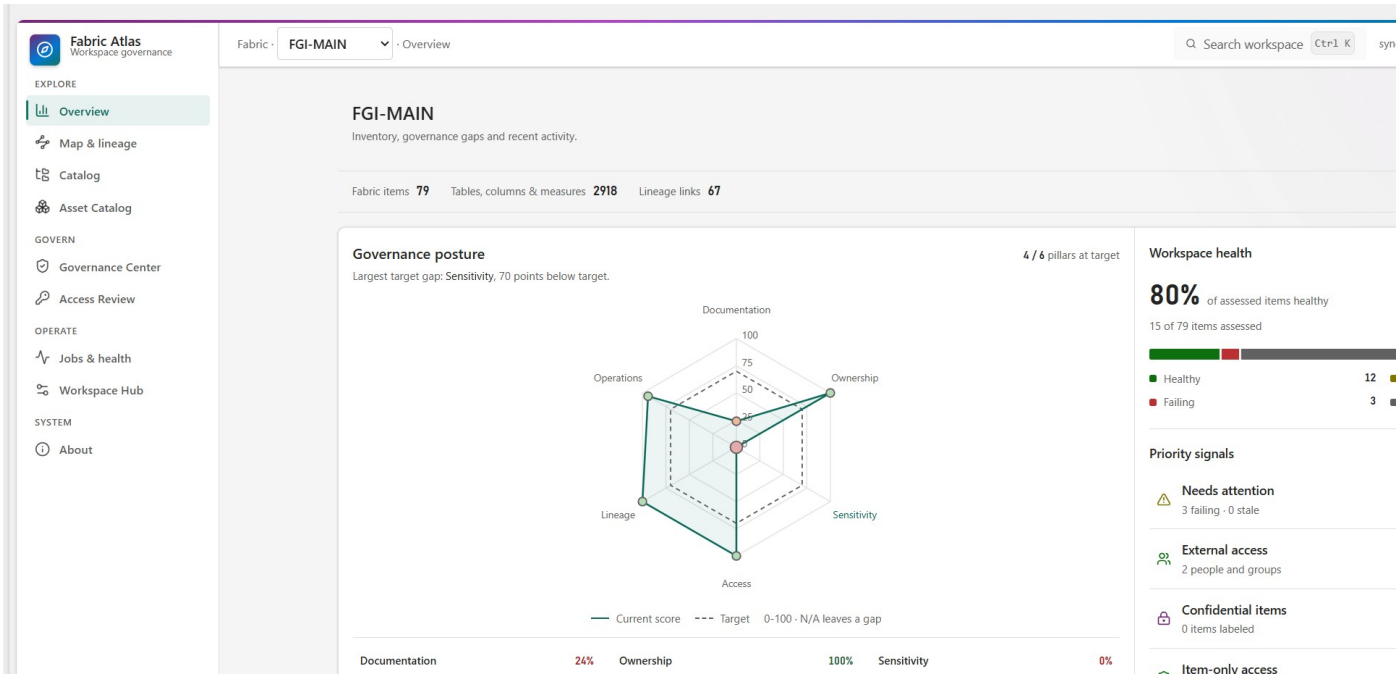
Fabric Atlas turns Fabric metadata into a shared working catalog.

Development, architecture and implementation choices

React + Rayfin Functions + Fabric SQL + minimal Python compatibility

Discover → Collect → Validate → Publish

One validated snapshot powers every workspace view.



Multi-workspace

Selected workspaces keep independent manifests and one active catalog.

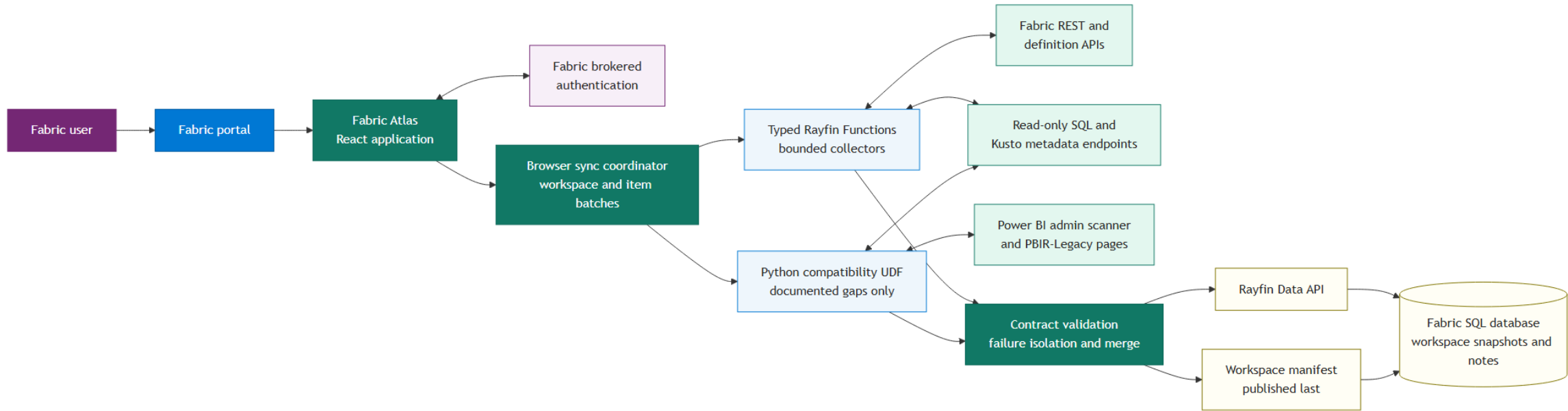
One evidence model

Catalog, lineage, access, governance and jobs read the same validated snapshot.

Metadata only

No business rows, notebook source, prompts, credentials or query text.

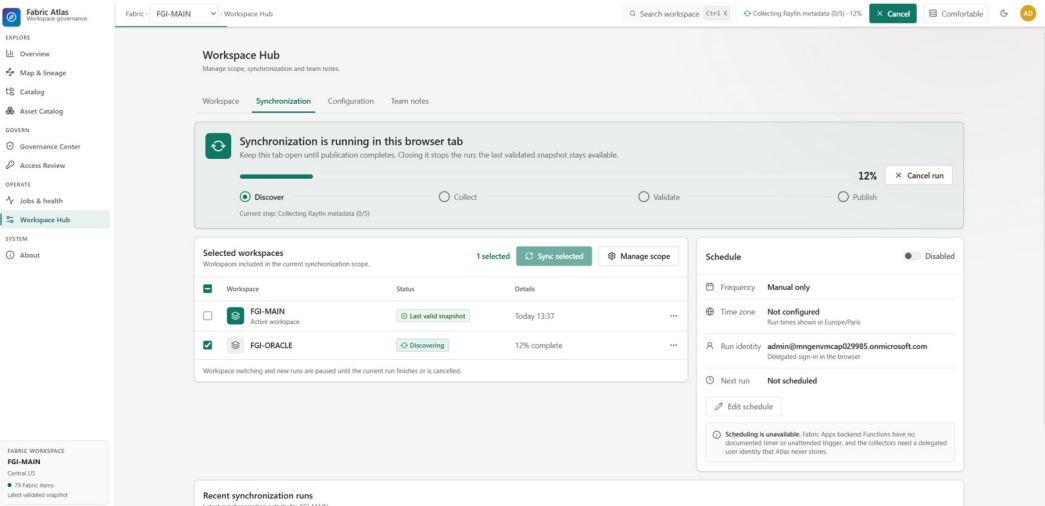
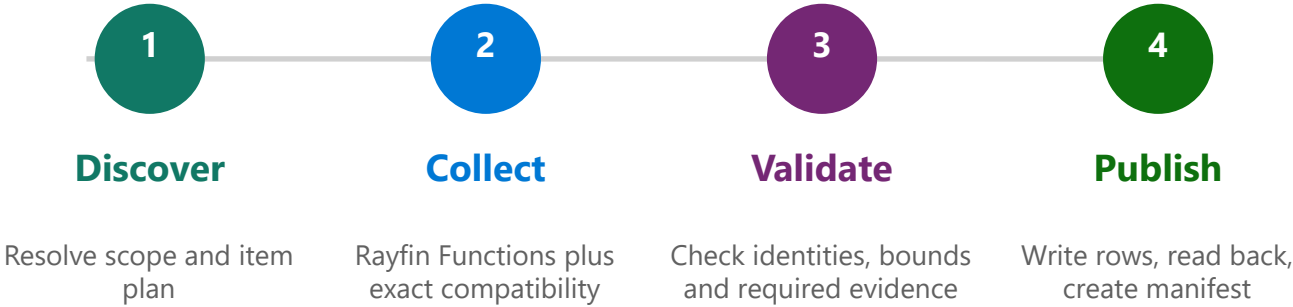
Rayfin Functions collect. The browser validates and publishes.



Trusted path Rayfin Functions → validated envelopes → Rayfin Data API → Fabric SQL

Compatibility Python runs only the exact Power BI, PBIR-Legacy and Kusto gaps.

One browser run, four stages, manifest last.



Last-known-good

Closing the tab stops future work. The previous validated snapshot remains visible.

Rayfin owns the normal path. Python is a bounded exception.

Rayfin Functions

Core inventory, roles and jobs

Definitions: Ontology, Graph Model, Data Agent

Power BI model and PBIR structure

Item Relations Preview

KQL structural metadata

SQL Database, Warehouse, Lakehouse and Mirrored Database catalogs

Shortcut, mirroring, MLV and policy evidence

Python compatibility

Power BI admin scanner

PBIR-Legacy pages

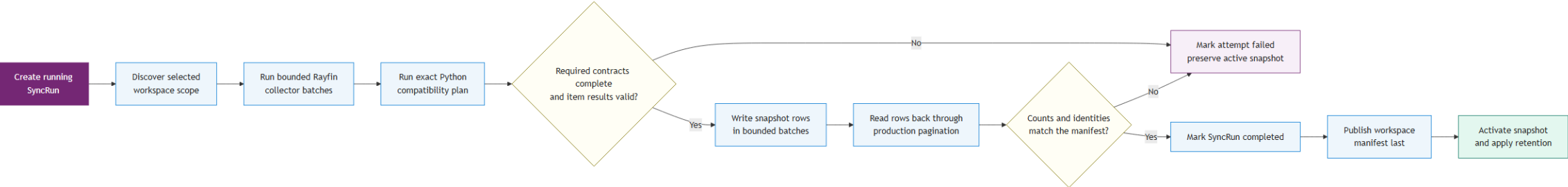
Kusto live schema

Explicit rollback

No workspace rediscovery

No unrequested collector

The manifest is the visibility switch.



1. Write candidate rows

All snapshot entities share the new workspace and snapshot identity.

2. Read back

Use the same paginated path as application startup and verify counts.

3. Publish manifest

Only the Workspace manifest makes the candidate visible.

Failure before the manifest keeps the previous snapshot active.

Twenty-six entity classes, five lifetimes.

Snapshot catalog | 7

Workspace, FabricItem, LineageEdge, Principal, AccessGrant, JobRun, ConfigEntry

Sync control | 7

SyncRun, SyncJob, SyncTask, SyncCommand, SyncRootRun, SyncPayloadManifest, SyncPayloadChunk

Shared control | 5

Comment, GovernancePolicy, GovernanceException, WorkspaceScope, SynchronizerAuthority

Personal state | 4

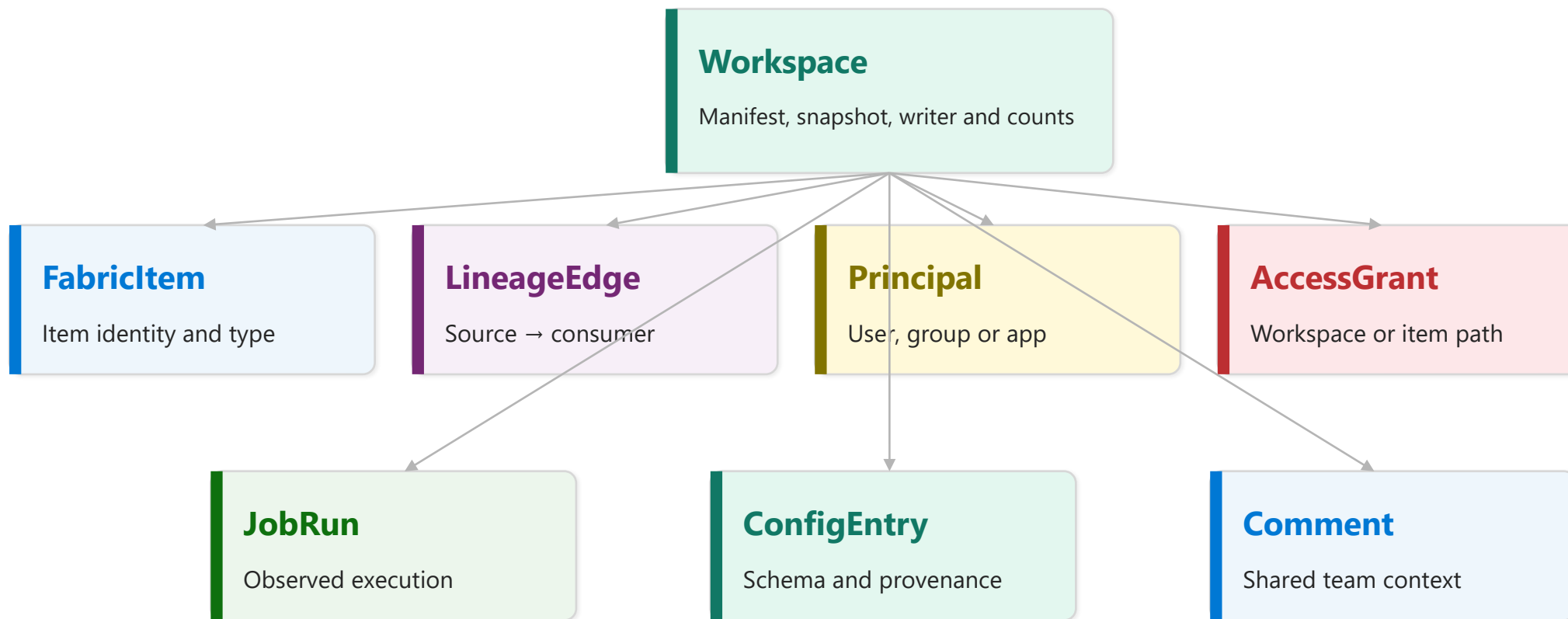
SavedView, AccessReview, AccessReviewEvent, FindingAck

Evidence stores | 3

ItemRelationsEvidenceSnapshot, AccessPolicyEvidence, OperationalIncident

Entity decorators define both the Fabric SQL schema and the authorization boundary.

Workspace and Fabric IDs connect snapshot evidence.



Twenty-six MSSQL tables grouped by lifetime.

Snapshot

Workspaces
FabricItems
LineageEdges
Principals
AccessGrants
JobRuns
ConfigEntries

Sync

SyncRuns
SyncJobs
SyncTasks
SyncCommands
SyncRootRuns
SyncPayloadManifests
SyncPayloadChunks

Shared

Comments
GovernancePolicies
GovernanceExceptions
WorkspaceScopes
SynchronizerAuthorities

Personal

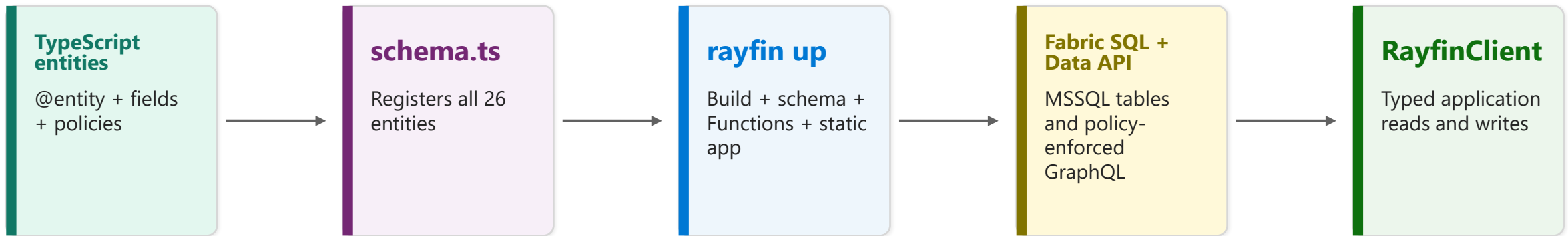
SavedViews
AccessReviews
AccessReviewEvents
FindingAcks

Evidence

ItemRelationsEvidence
Snapshots
AccessPolicyEvidences
OperationalIncidents

Logical links use stable workspace, snapshot, Fabric item and principal identities. Rayfin policies enforce the read and write boundary.

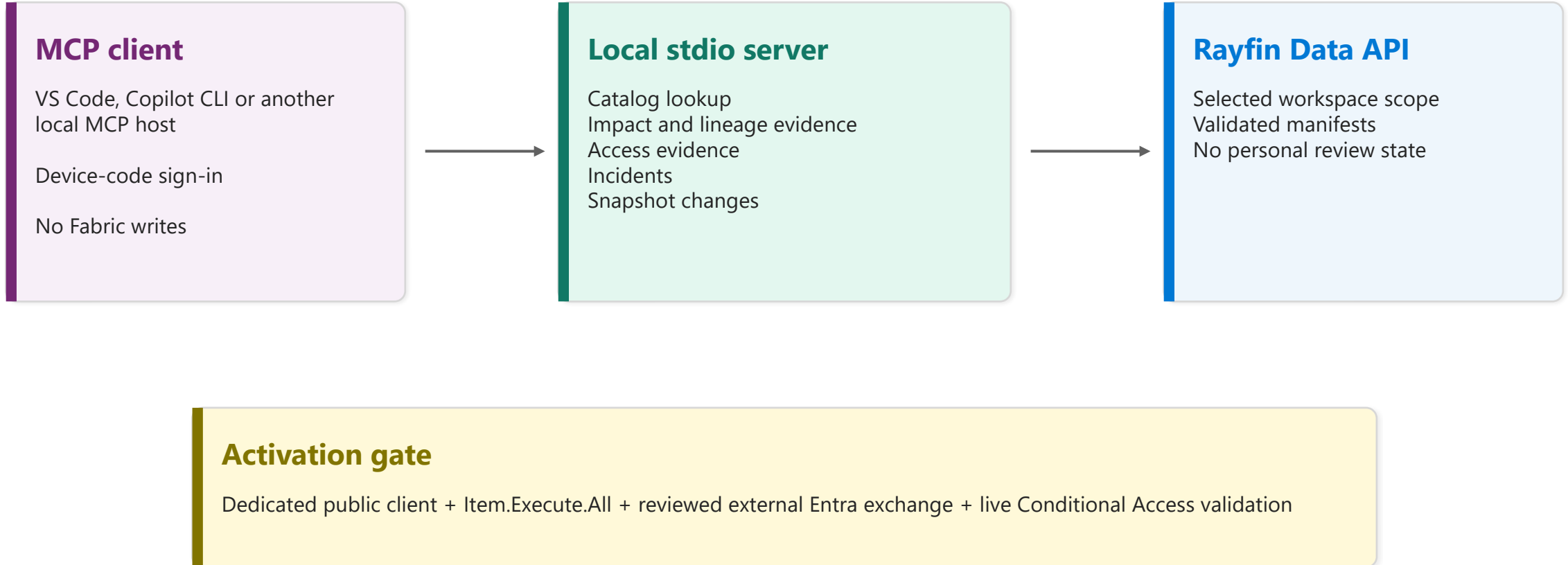
Decorated TypeScript becomes the Fabric data application.



Deployment invariant

Use the pinned Rayfin package family. Generated Function types and runtime metadata are never edited by hand.

The agent reads the same validated snapshots as the UI.



Two gates remain: remove Python, then remove the browser.

Remove Python compatibility

1. Power BI application audience or first-party scanner connector
2. PBIR-Legacy report-page coverage
3. Kusto application audience

Result: delete sync_compatibility

Run unattended

1. Timer or unattended trigger
2. Distributed claim or lease
3. Transactional publication fencing
4. Durable continuation after browser or worker interruption

Result: enable scheduled refresh

No browser tokens are stored or replayed.

The design keeps evidence useful without publishing partial state.

1

Generic scope

No tenant, workspace, item or endpoint hardcoding.

2

Bounded collection

Every collector has limits, evidence and failure isolation.

3

Explicit confidence

Verified, inferred, Beta and unavailable evidence remain distinct.

4

Manifest last

A snapshot is visible only after write, read-back and validation.

Deploy

```
npx rayfin up --tenant <tenant-id> --workspace-id <workspace-id> --item-name fabric-atlas --yes
```



FABRIC ATLAS

A trusted catalog stays available while the next snapshot is built.

Collect through bounded contracts. Model once. Publish after read-back.

Questions and feedback

github.com/fredgis/FabricAtlas